

Dossier del Selector de Recogida

Referencia completa del proyecto «Selector de punto de recogida» (app de Shopify) para el traspaso al equipo de producto y desarrollo: qué hace, de qué piezas se compone, dónde vive cada una, qué se configura y dónde, y el plan para sustituir los datos de demostración por los servicios web reales de Datadec.

Preparado para: product manager y equipo de desarrollo entrante **Fecha:** 18 de septiembre de 2026

Repositorio: github.com/rafher/pickup-point-selector

Estado: MVP funcional en tiendas de prueba, sin servicio real conectado todavía

RESUMEN EN 30 SEGUNDOS

Es una app de Shopify instalable por cualquier comercio de Datadec que añade un selector de punto de recogida en dos sitios: un **bloque de carrito** (cualquier plan) y una **extensión de checkout** (solo Shopify Plus). Ambos comparten un mismo backend en Node/Remix, con una base de datos Postgres para sesiones y ajustes por tienda. Los puntos de recogida hoy son **datos fijos de demostración** — el proyecto está preparado para enchufar el servicio web real sin tocar el bloque ni la extensión, cambiando solo la capa de datos del backend.

- Código: [repositorio en GitHub](https://github.com/rafher/pickup-point-selector)
- Backend en producción: `pickup-point-selector.onrender.com` (Render)
- App en Shopify: «Selector de punto de recogida», organización Datadec S.A.

Qué hace hoy / qué no hace

● Hace

● No hace (todavía)

- Bloque de carrito con selector de punto de recogida: vista de **lista** y vista de **mapa** (Leaflet + OpenStreetMap, sin API key).
- Extensión de checkout equivalente, visible solo cuando el comprador elige la tarifa de envío «Puntos» — vista de lista y mapa (con **Google Maps**, ver limitaciones).
- Ordena los puntos por **distancia** al código postal/ciudad que escribe el comprador (geocodificación aproximada propia, sin servicio externo).
- Guarda el punto elegido como **atributo del pedido**, visible en el Admin, igual venga del carrito o del checkout.
- Panel de ajustes en el Admin para que cada tienda configure la URL de su propio servicio de puntos (o siga usando los datos de demostración).
- Funciona de extremo a extremo hoy mismo con **datos de demostración fijos** (10 puntos repartidos por España).

- **No habla con ningún servicio real** de ninguna agencia de transporte — ver [plan de integración](#).
- **No resuelve varias agencias por tienda/mercado** — el diseño para eso está decidido pero no construido, ver [documento de multi-agencia](#).
- No hay una **Shopify Function** que genere la opción de envío «Puntos» dinámicamente — hoy es una tarifa de envío que cada comercio da de alta a mano en su Admin.
- No autorrellena el buscador de la extensión de checkout con la dirección real del comprador (exige un permiso de Shopify pendiente de solicitar, ver [limitaciones](#)).
- No está enviada a revisión ni distribuida a comercios reales — vive en dos tiendas de desarrollo.
- No tiene tests automatizados.

Arquitectura

Dos superficies de comprador, un backend compartido:

COMPRADOR · CARRITO

Bloque de tema

Liquid + JS plano,
Leaflet/OSM



MISMO ORIGEN

App Proxy de Shopify
/apps/pickup-points/*



BACKEND

proxy.points.tsx
authenticate.public.appProxy

COMPRADOR · CHECKOUT (PLUS)

Extensión de checkout

RED DIRECTA

BACKEND

api.checkout.points.tsx

Preact, componentes s-*,
Google Maps



fetch autenticado
token de sesión de checkout



authenticate.public.checkout
+ CORS

web/app/services/pickupPoints.server.ts

Punto único de integración — hoy sirve datos de demostración; mañana, el WS real de Datadec (ver plan de integración)

DATOS PROPIOS

Postgres (Neon)

Sesiones OAuth + ajustes por tienda

SOLO CHECKOUT

Google Maps

API key compartida de Datadec

Piezas del proyecto

El repositorio es un monorepo con tres partes independientes (no es un workspace de npm a propósito — ver [limitaciones](#)):

CARPETA	QUÉ ES	TECNOLOGÍA
web/	Backend + panel de ajustes embebido en el Admin	Remix (Node), Prisma, Polaris
extensions/pickup-point-picker/	Bloque de carrito (Theme App Extension)	Liquid, JS/CSS planos
extensions/pickup-point-checkout/	Extensión de checkout (Checkout UI Extension)	Preact + componentes web s-* de Shopify
docs/	Documentación de referencia (este dossier, el análisis de multi-agencia, otros apuntes)	HTML/PDF
render.yaml	Plantilla de despliegue en Render (no activa; el servicio actual se configuró a mano — ver despliegue)	Blueprint de Render
shopify.app.test-plugin.toml	Configuración de la app en Shopify — único fichero de config activo	Config as code del CLI

Backend (web/app/)

FICHERO	FUNCIÓN
routes/proxy.points.tsx	Endpoint que consume el bloque de carrito, vía App Proxy
routes/api.checkout.points.tsx	Endpoint que consume la extensión de checkout; añade CORS y la API key de Google Maps a la respuesta
routes/app._index.tsx	Pantalla de ajustes en el Admin (URL/API key del servicio real, vista por defecto)
routes/app.tsx	Layout del admin embebido (Polaris + App Bridge)

FICHERO	FUNCIÓN
<code>routes/auth.\$.tsx, routes/auth.login/</code>	OAuth de instalación
<code>routes/webhooks.app.uninstalled.tsx</code>	Limpia sesión y ajustes de una tienda al desinstalar
<code>services/pickupPoints.server.ts</code>	Punto único de obtención de puntos: decide demo vs. real, normaliza la respuesta externa, calcula y ordena por distancia
<code>services/pickupPoints.mock.ts</code>	10 puntos de demostración fijos (España)
<code>services/geo.server.ts</code>	Distancia (Haversine) y geocodificación aproximada propia (centroides de las 52 provincias)
<code>shopify.server.ts</code>	Configuración de @shopify/shopify-app-remix (OAuth, sesiones, webhooks)
<code>prisma/schema.prisma</code>	Modelos Session (OAuth) y ShopSettings (ajustes por tienda)

Dónde está desplegado

SERVICIO	PROVEEDOR	IDENTIFICADOR	PARA QUÉ
Código fuente	GitHub	<code>rafter/pickup-point-selector</code>	Repositorio único, rama main
Backend	Render (Free)	<code>pickup-point-selector.onrender.com</code>	API + Admin embebido; auto-despliega en cada push a main
Base de datos	Neon (Postgres)	<code>proyecto-pickup-point-selector</code> , región Londres (AWS eu-west-2)	Sesiones OAuth + ShopSettings
App de Shopify	Shopify Partners / Dev Dashboard	Org Datadec S.A. (id 211611850) · App «Selector de punto de recogida» (id 423273267201) · client_id 0249018a05a22cbdb144d9ffc8542599	Registro de la app, extensiones, versiones publicadas
Tienda de desarrollo — carrito	Shopify (plan Basic)	<code>datadec-dev-pickup.myshopify.com</code>	Probar el bloque de carrito (cualquier plan)
Tienda de desarrollo — checkout	Shopify (plan Plus)	<code>datadec-dev-extension.myshopify.com</code>	Probar la extensión de checkout (exige Plus, ver limitaciones)

Solo hay un fichero de configuración de app activo: `shopify.app.test-plugin.toml`. Hubo un `shopify.app.toml` original y un `shopify.app.datadec.toml` vinculado por error a otra app — ambos se eliminaron del repo; si aparecen en un clon antiguo, se ignoran.

Flujo paso a paso

Carrito (cualquier plan)

- 1 El comprador pulsa el campo «Punto de recogida»**
en la página del carrito.
- 2 El bloque abre un modal**
y pide los puntos a `/apps/pickup-points/points` — mismo dominio que la tienda, gracias al App Proxy, sin problemas de CORS.
- 3 El backend resuelve el origen**
(código postal/ciudad escrito por el comprador), calcula distancias y devuelve los puntos ordenados.
- 4 El comprador elige un punto y confirma**
— el bloque llama a `/cart/update.js` guardando dos atributos: `Punto de recogida` (texto legible) y `_pickup_point_id` (interno).

Checkout (solo Shopify Plus)

- 1 El comprador elige la tarifa de envío «Puntos»**
(dada de alta a mano por el comercio en Configuración → Envío y entrega).
- 2 La extensión detecta esa selección**
leyendo `shopify.deliveryGroups` y se muestra — permanece oculta con cualquier otra tarifa.
- 3 Pide los puntos**
a `api.checkout.points.tsx` directamente por red (no hay App Proxy en checkout), autenticando con el token de sesión de la propia extensión.
- 4 El comprador elige y confirma**
— se guardan los mismos dos atributos que en el carrito, vía `shopify.applyAttributeChange`.

Variables de configuración

Hay tres niveles distintos de configuración — no confundirlos es importante para el equipo que continúe:

1. Variables de entorno del backend (globales, un valor para toda la app)

En local: `web/.env`. En producción: panel de Render → el servicio → pestaña *Environment*.

VARIABLE	PARA QUÉ
SHOPIFY_API_KEY	= <code>client_id</code> de la app (no es secreto)
SHOPIFY_API_SECRET	Client secret — Dev Dashboard → app → Configuración → Credenciales
SCOPES	Vacío a propósito (la app no necesita permisos sobre datos de la tienda)
SHOPIFY_APP_URL	URL pública del backend (hoy, la de Render)
DATABASE_URL	Conexión Postgres agrupada (pooler) — la usa la app en marcha
DIRECT_DATABASE_URL	Conexión Postgres directa — la usa <code>prisma migrate</code> al desplegar
PICKUP_POINTS_API_URL / _API_KEY	Servicio real de puntos, a nivel global/por defecto — vacío hoy (usa demo). Cada tienda puede tener su propio valor, ver abajo
GOOGLE_MAPS_API_KEY	Key compartida de Datadec para el mapa de la extensión de <code>checkout</code> (el bloque de carrito no la necesita, usa <code>OpenStreetMap</code>)

2. Ajustes por tienda (cada comercio instalado configura los suyos)

Se editan desde el propio Admin de cada tienda → la app → pantalla de ajustes (`web/app/routes/app._index.tsx`), y se guardan en la tabla `ShopSettings` de la base de datos. Tienen prioridad sobre las variables de entorno globales.

CAMPO	PARA QUÉ
<code>pickupApiUrl</code>	URL del servicio de puntos de esa tienda, si es distinto del global
<code>pickupApiKey</code>	API key de ese servicio (se envía como <code>Authorization: Bearer</code>)
<code>defaultView</code>	<code>list</code> o <code>map</code> — vista inicial del bloque de carrito

3. Ajustes visuales del bloque de carrito (por tema, sin tocar código)

Editor de temas de cada tienda → bloque «Punto de recogida» → panel de ajustes del bloque (definidos en el `{% schema %}` de `pickup-point-picker.liquid`).

CAMPO	POR DEFECTO
Etiqueta, placeholder, título del modal, texto de confirmación	«Punto de recogida», «Selecciona un punto de recogida», «Elige tu punto de recogida», «Confirmar punto de recogida»
Textos de las pestañas Lista/Mapa, buscador, error	«Lista», «Mapa», «Tu código postal o ciudad», mensaje de error genérico
Vista inicial, color de acento, nombre del atributo del pedido	list, #1a1a1a, «Punto de recogida»

A TENER EN CUENTA La extensión de **checkout** no tiene panel de ajustes por tema — sus textos están en un diccionario dentro del propio código (`Checkout.jsx`), y la URL del backend está **hardcodeada** como constante (`BACKEND_URL`). Cambiar cualquiera de las dos cosas exige editar código y volver a desplegar la extensión, no es autoservicio para el comercio todavía.

Cómo desplegar cambios

Backend (web/)

Automático: al hacer `git push` a `main`, Render reconstruye y despliega solo (build: `npm install && npx prisma generate && npx prisma migrate deploy && npm run build`; arranque: `npm run start`).

Extensiones (bloque de carrito + extensión de checkout)

Manual, desde la raíz del repo:

```
npx shopify app deploy --allow-updates
```

Publica una nueva versión de la app en Shopify con el contenido actual de ambas carpetas de `extensions/`. No hace falta reinstalar la app en las tiendas para que los cambios de **código** de una extensión ya publicada lleguen — sí hace falta reinstalar cuando cambia algo de **registro** de la app (App Proxy, capacidades como `network_access`) — ver [limitaciones](#).

EVITAR No usar `shopify app dev` para el día a día: levanta un túnel temporal y, por tener `automatically_update_urls_on_dev = true`, puede sobrescribir `application_url/app_proxy.url` del `.toml` con la URL de ese túnel en vez de la de Render. Solo usarlo a propósito, sabiendo que luego hay que volver a dejar esas URLs apuntando a producción antes del siguiente `deploy`.

Accesos y credenciales

Por seguridad, este documento no incluye contraseñas ni claves — solo dónde están.

Cuenta de Shopify Partners

QUIÉN LA TIENE

Fernando Izquierdo (propietario de la organización Datadec S.A. en Partners)

QUÉ DA ACCESO A

Dev Dashboard de la app, Partner Dashboard clásico, las dos tiendas de desarrollo

Render

DÓNDE

dashboard.render.com → workspace Datadec → servicio `pickup-point-selector`

QUÉ HAY AHÍ

Variables de entorno, logs de despliegue, historial de deploys

Neon (base de datos)

DÓNDE

console.neon.tech → proyecto `pickup-point-selector`

QUÉ HAY AHÍ

Cadenas de conexión (agrupada y directa), rama `production`

Contraseñas de escaparate

DÓNDE

Admin de cada tienda de desarrollo → Tienda online → Preferencias

NOTA

Son dos contraseñas distintas, una por tienda — no la misma

Limitaciones y comportamientos de plataforma a tener en cuenta

Cosas descubiertas construyendo este MVP que le ahorrarán tiempo al equipo entrante:

- **Node fijado a 20.x** (`web/.node-version` + `engines` en `package.json`): Render resolvía por defecto una versión demasiado nueva que colgaba el build sin dar error.
- **`web/` no es un workspace de npm a propósito**: serlo hacía que Render instalara las dependencias en el `node_modules` de la raíz del repo en vez del de `web/`, y el comando de build no encontraba `remix`. Para desarrollo local: `npm install` en la raíz (CLI) y `cd web && npm install` aparte.
- **Acceso de red de la extensión de checkout** (`network_access`): hace falta aprobarlo una vez en el Partner Dashboard clásico (Apps → API access →

«Permitir acceso de red...») — el Dev Dashboard nuevo no tiene ese control todavía (hueco conocido de su migración).

- **Datos protegidos del comprador (Nivel 2):** la extensión de checkout no puede leer `shopify.shippingAddress` sin que Shopify apruebe ese acceso — proceso de revisión, no instantáneo. Por eso el buscador de la extensión es manual, no autorrellenado. Si se solicita y se concede, se puede retomar el autorrelleno (código ya explorado, revertido por ahora).
- **Las tiendas de desarrollo «Basic» no muestran los bloques de checkout Plus-only en el editor**, aunque el plan de Partners diga «Developer Preview». Hay que crear la tienda de desarrollo eligiendo explícitamente «Shopify plan: Plus» al crearla.
- **Tras cambiar el App Proxy o las capacidades de una extensión**, a veces la tienda necesita **reinstalar la app** (desinstalar + volver a instalar) para que el cambio de registro se refleje — no está documentado por Shopify, se detectó empíricamente.
- **Google Maps API key compartida:** al ser una sola key de Datadec para todas las tiendas instaladas, hay que ir añadiendo el dominio de cada cliente nuevo a sus restricciones de referencia HTTP en Google Cloud Console — tarea de mantenimiento recurrente, no automática.
- **La opción de envío «Puntos» es manual, no una Shopify Function:** cada comercio tiene que darla de alta a mano en cada zona de envío donde quiera ofrecer recogida. La extensión de checkout la detecta comparando el *título* de la tarifa elegida (configurable en `PICKUP_DELIVERY_OPTION_TITLE` dentro de `Checkout.jsx`), no por un identificador estable.

Plan: conectar con los servicios web reales de Datadec

Hoy `getPickupPoints()` (en `pickupPoints.server.ts`) es el único punto de la aplicación que sabe de dónde vienen los puntos — ni el bloque de carrito, ni la extensión de checkout, ni las rutas que los exponen necesitan cambiar cuando se conecte el servicio real.

— ver el documento dedicado (enlace abajo). Condiciona la forma exacta del contrato del paso siguiente si un mismo comercio puede tener varias agencias.

2 Conseguir el contrato real del WS

del equipo de infraestructuras de Datadec: URL base, autenticación (hoy se asume `Authorization: Bearer <api key>`, a confirmar), forma exacta de la petición y la respuesta, límites de uso/caché recomendados, formato de errores.

3 Ajustar `normalizeExternalResponse()`

en `pickupPoints.server.ts` para traducir la forma real del WS a `PickupPoint` — hoy admite de forma laxa nombres de campo en inglés o español como punto de partida razonable, pero hay que ajustarlo al contrato real exacto.

4 Configurar `PICKUP_POINTS_API_URL/_API_KEY`

(global en Render, o por tienda desde el Admin) apuntando al WS real — en cuanto haya valor, deja de usarse el mock automáticamente, sin más cambios de código.

5 Probar de extremo a extremo

en una tienda de desarrollo, comprobando en particular: tiempos de respuesta del WS real bajo el modal (hoy no hay caché ni timeout propios — a valorar si hace falta añadir alguno), y qué pasa si el WS falla (hoy cae automáticamente al listado de demostración, a decidir si ese es el comportamiento deseado en producción o si debería mostrar un error explícito).

6 Revisar las decisiones pendientes

del documento de multi-agencia (cómo le llega el país al WS, si hace falta un filtro de agencia en la UI) antes de dar el WS real por cerrado.

Documentos relacionados

- [README del repositorio](#) — puesta en marcha local, arquitectura del backend con más detalle

- [Puntos de Recogida Multi-Agencia](#) — documento de decisión sobre cómo modelar varias agencias de transporte por tienda/mercado

Dossier preparado a partir del desarrollo del MVP del proyecto «Selector de punto de recogida». Refleja el estado del repositorio a 18 de septiembre de 2026 — verificar contra el código antes de dar por buena cualquier ruta o nombre de fichero citado aquí.